



Turbine Digital-Twin Dashboard Research Report

The strongest solution is a **hybrid ThingsBoard dashboard**: a custom Babylon.js 3D turbine widget as the visual centerpiece, an authoritative SVG/SCADA view as the operational fallback, and native ThingsBoard charts, alarms, aliases, actions, and dashboard states for diagnostics. No inspected open-source project provides a production-ready ThingsBoard steam-turbine digital twin, but several projects solve important pieces—most notably ThingsBoard’s native SVG SCADA framework, the FUXA-based community widget, and a working Babylon.js telemetry-widget example.

Executive recommendation

Build one seamless, full-screen “digital-twin workspace” rather than a collection of gauges:

1. **Primary visualization:** Babylon.js custom ThingsBoard widget displaying a named-node GLB model of the confirmed equipment—ESV, TV1, TV2, turbine casing, wheel case, intermediate GBC location, gearbox, dyno, inlet, exhaust and three leakage paths.
2. **Operational fallback:** A polished SVG process mimic derived directly from the attached schematic, implemented with native ThingsBoard SCADA symbols or the FUXA-based community renderer.
3. **Contextual telemetry:** Display only 8–12 critical values persistently; expose the remaining channels as hover labels, selectable markers and component-specific panels.
4. **Drill-down:** Use dashboard states for overview, process, turbosisory, component, sensor and historical-replay views. ThingsBoard officially supports state-based navigation through widget actions.^[1]
5. **Diagnostics:** Use native ThingsBoard time-series and alarm widgets beside or beneath the visual twin rather than rebuilding charting and alarm management inside WebGL.
6. **Model strategy:** Prefer project CAD or a purpose-built simplified Blender model. No exact steam-turbine GLB asset with sufficiently clear permissive licensing and matching geometry was verified; a random marketplace turbine would be visually misleading.

The 3D widget is realistic to implement. The critical constraint is not WebGL—it is obtaining a trustworthy model, establishing sensor-to-component metadata, validating alarm thresholds, and managing lifecycle, asset loading and performance correctly.

Source-file findings

The CSV contains 3,589 observations and 62 columns: one timestamp plus 61 telemetry channels. All telemetry columns are numeric and populated throughout the supplied dataset.^[2]

The attached schematic establishes the following confirmed equipment topology:

Inlet steam → actuator/ESV → TV1/TV2 → turbine → gearbox → dyno, with wheel-case, intermediate-GBC, exhaust and leakage measurement branches. The second view establishes two axial turbine channels, four radial turbine channels and four radial gearbox channels.^[3]

Important data issues

- The CSV timestamps span approximately one hour, but values are stored only to minute precision even though roughly 3,589 samples exist. Ingesting these timestamps literally would assign many samples the same timestamp; obtain the original second- or millisecond-resolution DAQ timestamp before implementing playback.^[2]
- The PDF says torque is in N.m, while GB_TRQ in the CSV is in kN.m. Do not convert or calculate power until the instrumentation definition confirms which unit is authoritative.^{[3] [2]}
- The PDF and CSV use `mills`. This may mean vibration displacement in `mils`, but the UI should retain the source label until instrumentation documentation confirms the intended unit.^{[2] [3]}
- The CSV has no explicit turbine or dyno power channel even though power placeholders appear in the schematic.^{[3] [2]}
- TURBINE_SPEED and GB_TRQ may refer to different shaft sides. Mechanical power must not be calculated from them without confirming gearbox ratio and torque-measurement location.
- The vibration channels are scalar values. Without phase, waveform or synchronous reference data, the dashboard cannot legitimately produce shaft-orbit plots or true vibration waveforms.
- Alarm limits cannot be inferred safely from the observed minima and maxima. Store approved limits as ThingsBoard attributes derived from test specifications, probe range and machine limits.

Confirmed sensor map

Process instrumentation

Confirmed area	PDF label	CSV key	Display treatment	Confidence
Inlet steam	PT109	PT_109A	Persistent inlet pressure	High; suffix differs
Inlet steam	TT109	TT_109A	Persistent inlet temperature	High; suffix differs
Inlet steam	FT110A	FT_110A	Persistent mass flow; drives inlet-flow animation	High
ESV	PT111B	PT_111B	Hover label near ESV	High
TV1	PT111	PT_111	Hover label beside TV1	High
TV1	TT111	TT_111	Hover label beside TV1	High
TV2	PT112	PT_112	Hover label beside TV2	High
TV2	TT112	TT_112	Hover label beside TV2	High
Leakage 3, upstream/left	PT160	PT_160	Hover/click leakage branch	High
Leakage 3, upstream/left	TT160	TT_160	Hover/click leakage branch	High
Leakage 2	PT161	PT_161	Hover/click leakage branch	High
Leakage 2	TT161	TT_161	Hover/click leakage branch	High
Leakage 1	PT162	PT_162	Hover/click leakage branch	High
Leakage 1	TT162	TT_162	Hover/click leakage branch	High
Leakage 1	FT162	FT_162	Flow label; can drive branch animation	High
Wheel case	PT120	PT_120	Component detail	High
Wheel case	TT120/R	TT_120_R	Component detail	High
Wheel case	Not separately shown	TT_120	Keep in detail panel pending clarification	Low
Intermediate GBC	PT111C	PT_111C	Component detail	High
Intermediate GBC	TT111C/R	TT_111C, TT_111C_R	Combined or redundant pair	Medium
Exhaust	PT150A/B	PT_150A, PT_150B	Pair summary plus individual details	High
Exhaust	TT150A/B	TT_150A, TT_150B	Pair summary plus individual details	High
Leakage 3, downstream/right	PT163	PT_163	Hover/click downstream branch	High
Leakage 3, downstream/right	TT163	TT_163	Hover/click downstream branch	High

The schematic uses “Leakage 3” for two visibly separate branches. Retain distinct internal identifiers such as leakage_3_upstream and leakage_3_downstream, but do not rename the physical process labels until the test-rig documentation resolves the duplication. ^[3]

Vibration placement

Equipment/station	Orientation	PDF label	CSV key
Turbine casing, axial top	Axial	ZT600	ZT_600
Turbine casing, axial bottom	Axial	ZT601	ZT_601
Turbine radial station 1	X axis	XT600	XT_600
Turbine radial station 1	Y axis	XT601	XT_601
Turbine radial station 2	X axis	XT602	XT_602
Turbine radial station 2	Y axis	XT603	XT_603
Gearbox radial station 1	X axis	XT604	CSV XT 604
Gearbox radial station 1	Y axis	XT605	CSV XT 605
Gearbox radial station 2	X axis	XT606	XT_606
Gearbox radial station 2	Y axis	XT607	CSV XT 607

This ordering and orientation come directly from the turbovisory page and should determine marker placement. ^[3]

Performance and control

Key	Recommended placement
TURBINE_SPEED	Permanent hero KPI; drives symbolic shaft rotation
GB_TRQ	Permanent KPI between turbine and gearbox/dyno
HP_DEMAND	Control-demand overlay near actuator
ACT_POS_FB	Actuator feedback beside ESV/actuator
PYRO_T	Turbine temperature condition marker
PYRO_GB	Gearbox temperature condition marker
Dyno power	Show "Not instrumented" until a verified key or calculation exists
Dyno speed	Do not reuse TURBINE_SPEED without confirming shaft relationship

Unlocated channels

The PDF does not establish defensible physical locations for:

- PT_110A, PT_110B
- PT_153
- PT_201, PT_253
- TT_110A, TT_110B
- Dyno Water O/L
- RTD_219A, RTD_219B
- RTD_220, RTD_221
- RTD_200–RTD_205

These should initially appear in an **Auxiliary and unassigned instrumentation** panel. Moving them onto the 2D or 3D machine before obtaining the P&ID, loop schedule or sensor-location drawing would create false engineering information.

ThingsBoard feasibility

Proven platform capabilities

Current ThingsBoard custom widgets support HTML, scoped CSS, JavaScript, external JavaScript/CSS resources and lifecycle callbacks such as `onInit`, `onDataUpdated`, `onResize` and `onDestroy`. The current official development guide also describes widget context access, action sources and multiple widget types through the [ThingsBoard widget-development documentation](#).

ThingsBoard's current SCADA framework uses SVG resources with tagged SVG elements, configurable properties, behaviors and render functions. This makes native SVG the lowest-risk route for interactive equipment coloring, valve state, flow animation and clickable sensors.^[4]

Dashboard states provide supported multi-level navigation, and actions can navigate to component- or entity-specific states. Standard action buttons can navigate, open URLs or execute custom JavaScript.^[5] ^[1]

Dashboard configurations can be exported and imported as JSON through the current [dashboard documentation](#). This should be used for version-controlled dashboard delivery, but external GLB/SVG assets and custom widget dependencies must also be packaged and documented separately.

3D evidence

A working community tutorial in [blackstar-baba/how-2-use-thingsboard](#) demonstrates Babylon.js inside a ThingsBoard widget. It creates a canvas and Babylon engine, attaches a camera, links a GUI label to a mesh, updates that label from live subscription data, resizes the engine and renders continuously.

That example targets ThingsBoard 3.3 and Babylon.js 4.2, so it proves the architectural technique—not current plug-and-play compatibility. Its advice to concatenate dependent scripts addressed the older asynchronous resource-loading behavior; for a new implementation, bundle the selected Babylon version, GUI and glTF loader into one pinned build.

Feasibility classification

Capability	Classification	Notes
Interactive SVG mimic	Proven in ThingsBoard	Native SCADA SVG support
Telemetry-driven SVG styles	Proven	Render behavior and data bindings

Capability	Classification	Notes
Babylon canvas in widget	Previously demonstrated	Revalidate on the deployed TB version
Live 3D labels	Previously demonstrated	Babylon GUI linked to mesh
glTF/GLB loading	Likely	Normal Babylon capability; model hosting/CORS must be tested
Three.js widget	Technically likely	No equally strong inspected ThingsBoard-specific implementation
WebGL picking	Likely	Use Babylon pointer observables or action managers
Dashboard drill-down	Proven	Widget actions and dashboard states
External 3D iframe	Possible	Authentication, CSP and communication complicate integration
Full CAD/BIM viewer inside widget	Difficult	Bundle size, memory and licensing
Historical 3D playback	Moderate to difficult	Requires synchronized time-series retrieval and a replay controller

ThingsBoard does not document a general prohibition on WebGL. The real integration risks are dependency loading, content-security policy, CORS, WebGL context loss, multiple widget instances, and failure to dispose the engine in `onDestroy`.

Existing ThingsBoard resources

Best widgets

Rank	Resource	What it provides	Quality	Difficulty	Directly reusable
1	Native SCADA symbols	SVG symbols, tags, behaviors and render functions	High	Moderate	Yes, subject to edition/version
2	TB SCADA Widget	FUXA-derived SVG renderer, industrial symbols and telemetry adapter	Promising prototype	Moderate-high	Build/verify first
3	HTML Value Card	Multi-key live HTML/CSS overlays	High	Easy	Yes ^[6]
4	Native time-series chart	Historical and real-time trends	High	Easy	Yes
5	Native alarm widgets	Severity, acknowledgement and alarm lifecycle	High	Easy	Yes ^[7]
6	Native action button	Navigation and scripted interaction	High	Easy	Yes ^[5]

Community repositories

Rank	Repository	Evidence and relevance	License	Reuse status
1	foreverCP3/tb-scada-widget	SCADA/HMI widget powered by a FUXA renderer; README claims 40+ symbols, bindings and flow/blink/color/rotation animation	Apache-2.0 stated	Source available; validate build
2	devaskim/awesome-thingsboard	Curated widgets, dashboard exports, snippets and tools; 209 stars and 37 forks at inspection	Verify per linked project	Catalog, not one import
3	TheXiong/meta2d-thingsboard-designer	Designer integration listed by the curated ThingsBoard catalog	Verify	Promising 2D authoring route
4	rossmann-engineering/Collibellx-Thingsboard-Widgets	Widget library advertised for ThingsBoard CE/PE 3.0+	Verify	Components rather than turbine twin
5	p4hmartins1/conversor-svg-thingsboard	Adds ThingsBoard-style tags to SVG for SCADA symbols	Verify	Utility requiring review
6	AbCthings/thingsboard-widgets	Community chart and gauge widgets	Verify	Useful for secondary telemetry
7	thingsboard-widget	Community inter-widget communication technique	Verify	Pattern/reference only

The [TB SCADA Widget README](#) claims an importable `dist/tb-industrial-widgets-bundle.json`, but the inspected main-branch root did not contain a `dist` directory. Treat it as a source project until a release artifact or reproducible build is confirmed.

The curated [Awesome ThingsBoard README](#) is valuable for discovery but contains few machinery-specific widgets. It does not reveal a mature, reusable turbine dashboard.

Technology scoring

Scores use 1–5, where 5 is strongest. “Effort” is reverse-scored: 5 means lowest effort. Scores are project-specific judgments, not vendor benchmarks.

Approach	Visual	3D	TB integration	Live data	Hover/click	Overlays	Drill-down	Animation	State color	Performance	Maintainability	Effort	Browser
Standard widgets	2	1	5	5	3	2	4	2	4	5	5	5	5
Custom native SVG	4	1	5	5	5	5	5	4	5	5	4	3	5
Canvas/WebGL custom	4	3	3	5	4	4	4	5	5	4	3	2	4
Three.js widget	5	5	3	5	5	5	4	5	5	4	3	2	4
Babylon.js widget	5	5	4	5	5	5	4	5	5	4	4	2	4
PlayCanvas widget	5	5	2	4	5	4	3	5	5	4	3	2	4
External 3D app	5	5	2	3	5	5	4	5	5	4	4	2	4
SVG + 3D hybrid	5	5	4	5	5	5	5	5	5	4	4	1	4
FUXA/native SCADA + 3D	5	5	4	5	5	5	5	5	5	4	3	1	4

Best overall strategies

Rank	Strategy	Verdict
1	Native SVG SCADA plus Babylon 3D plus native diagnostics	Best balance of visual impact, credibility and resilience
2	Native SVG SCADA only	Best MVP and best operational reliability
3	Babylon 3D plus native charts	Strongest single-view demonstration
4	FUXA-based SCADA plus optional 3D	High reuse potential, but community project requires hardening
5	External 3D application embedded in ThingsBoard	Best for frontier features, weaker deployment simplicity
6	Standard ThingsBoard widgets	Suitable diagnostics layer, unsuitable as the primary experience

3D library ranking

Rank	Technology	Strength for this project	ThingsBoard status	Modification required
1	Babylon.js	Engine, glTF loading, picking, GUI mesh labels, materials, animation and clipping	Older working example exists	Build a pinned bundle and TB adapter
2	Three.js	Largest web-3D example ecosystem and flexible rendering	Likely, not specifically proven by inspected project	Add CSS labels, picking and state layer
3	model-viewer	Simplest GLB presentation and camera controls	Likely	Considerable custom work for 60 live sensors
4	PlayCanvas Engine	Polished real-time 3D and editor workflow	Technically possible	Heavier integration and deployment workflow
5	VTK.js	Engineering/scientific mesh and scalar visualization	Possible	Excessive for the initial dashboard
6	External Unity/Unreal/Web app	Highest graphical ceiling	Better external	Separate deployment, authentication and messaging

Babylon.js ranks first because the ThingsBoard-specific example already demonstrates a live mesh-linked label. Three.js remains a reasonable alternative if the team has stronger Three.js experience.

2D/SCADA ranking

Rank	Approach	Relevance	Difficulty	Quality
1	Native ThingsBoard interactive SVG	Supported bindings and navigation, minimal integration risk	Moderate	High
2	Purpose-built SVG with custom widget logic	Complete control over the exact test-rig topology	Moderate	Very high

Rank	Approach	Relevance	Difficulty	Quality
3	FUXA-based TB SCADA widget	Reusable symbols and industrial animations	Moderate-high	Promising
4	Meta2D ThingsBoard designer	Low-code process-mimic authoring	Moderate	Requires validation
5	Static schematic with HTML sensor overlays	Fastest prototype	Easy	Medium
6	Conventional cards and gauges	Useful only as diagnostics	Easy	Low as primary screen

SVG may be more operationally professional than 3D because labels remain legible, branches are unambiguous and all relevant components can stay visible. The optimal design therefore provides **Twin mode** and **Process mode**, not a forced choice between them.

Model and symbol assets

3D model sources

Rank	Source	License confidence	Suitability
1	Existing project CAD exported through Blender to GLB	Highest if project-owned	Best geometry and component identity
2	Purpose-built simplified Blender model	Project-controlled	Best practical option if CAD is unavailable
3	Individually licensed Sketchfab/CGTrader asset	Asset-specific	Useful only after geometry and licence review
4	GrabCAD model	Upload-specific and often restrictive	Reference/CAD starting point only
5	Generic glTF sample libraries	Usually clear but not turbine-specific	Loader testing only

No exact model should be recommended merely because it is downloadable. A visually impressive but geometrically unrelated turbine would undermine the instrumentation mapping.

SCADA symbols

Rank	Source	What to reuse
1	ThingsBoard native SCADA resources	Valves, piping, indicators, state behaviors
2	FUXA and its TB adapter	Pipes, valves, instruments, animation patterns
3	Existing PDF geometry redrawn as SVG	Authoritative turbine, GB, dyno and leakage topology
4	Project-specific SVG symbols	ESV, TV1/TV2, probe markers and turbovisory stations
5	Generic icon sites	Minor UI icons only; verify every asset licence

Create the turbine casing and shaft geometry specifically for this dashboard. Generic symbol libraries are most useful for valves, pipes, transmitters and alarm indicators.

Turbine visual design

Primary composition

Use a dark, borderless workspace with the machine occupying approximately 70% of the screen:

- Inlet and control equipment on the left.
- Turbine in the center.
- Gearbox and dyno on the right.
- Leakage and exhaust routes beneath the shaft line.
- A narrow condition rail across the top.
- A collapsible diagnostics drawer along the bottom or right edge.
- No grid of rectangular KPI cards around the machine.

The machine should remain neutral gunmetal or desaturated blue during normal operation. Color should carry meaning:

- Normal: subtle green marker, neutral equipment body.
- Warning: amber outline and marker.
- Alarm: red component edge/emissive accent.
- Critical: red pulse plus focused camera and alarm banner.
- Stale data: grey/hatch treatment.

- Bad-quality or disconnected data: purple or crossed marker.

Do not paint the entire screen green during normal operation.

Persistent telemetry

Keep these values permanently visible:

- Turbine speed.
- GB torque.
- HP demand.
- Actuator position feedback.
- Inlet pressure, temperature and mass flow.
- Exhaust pressure and temperature summary.
- Maximum vibration severity.
- PYRO turbine and gearbox temperatures.
- Overall alarm state and data freshness.

All other measurements should appear through hover, selection or component expansion.

Interaction model

Hovering over a marker

- Enlarges the marker.
- Highlights its leader line and associated component.
- Shows ID, value, unit, quality, timestamp and severity.
- Temporarily shows a small sparkline if sufficient history is loaded.

Clicking a sensor

- Locks the selection.
- Opens the sensor diagnostics drawer.
- Shows current value, alarm limits, recent trend, minimum/maximum, related sensors and latest alarms.
- Provides “Open full history” and “Locate on machine.”

Clicking a component

- Moves the camera to a preset.
- Fades unrelated equipment.
- Reveals the component’s sensor markers.
- Navigates or synchronizes the associated dashboard state.

Double-click or expand

- Opens a dedicated component state such as turbine vibration or gearbox condition.

Dashboard states

State	Purpose	Main content
overview	Immediate rig health	3D/2D twin, persistent KPIs, alarm ribbon
process	P&ID-style operation	Full SVG process mimic and leakage paths
turbine	Turbine condition	Casing, inlet/outlet stations, temperatures and vibration
gearbox	Gearbox condition	GB temperatures, torque and XT604–XT607
turbovisory	Vibration overview	Probe arrangement, X/Y pairs and trends
sensor	Individual channel	Latest value, history, limits, quality and alarms
performance	Test performance	Speed, demand, actuator response, torque, flow and validated power
replay	Historical reconstruction	Time slider, synchronized machine state and charts
auxiliary	Unmapped channels	PT110A/B, PT153, PT201/253, dyno-water and RTDs

The turbovisory state should reproduce the confirmed physical ordering from the PDF, not reorganize channels numerically. ^[3]

3D implementation

Model structure

Export one optimized GLB with named nodes resembling:

```
rig_root
  inlet_pipe
  esv
  tv1
  tv2
  turbine_casing
  turbine_shaft
  wheel_case
  gearbox
  dyno
  exhaust_path
  leakage_1
  leakage_2
  leakage_3_upstream
  leakage_3_downstream
```

Only include components confirmed by the schematic. Internal blade stages, bearing arrangements and cutaway geometry require CAD or engineering confirmation.

Keep sensor markers outside the GLB in a configuration object:

```
sensorId
sourceKey
displayName
units
system
componentNode
position3D
position2D
orientation
displayTier
warningLimit
alarmLimit
criticalLimit
```

Threshold values should reference ThingsBoard attributes rather than be compiled into the widget.

Widget lifecycle

- **onInit:** Create engine, scene, camera, lights and overlay layer; load the GLB and metadata.
- **onDataUpdated:** Convert subscription data to a latest-value map; update labels, severity, component state and animations.
- **onResize:** Resize the Babylon engine and recompute label layout.
- **onDestroy:** Stop the render loop, remove observers and event handlers, dispose textures, scene and engine.
- **Widget actions:** Expose sensor-click and component-click action sources to ThingsBoard where practical.
- **Dashboard state integration:** Keep the selected sensor/component in state parameters or a small selection service.

Performance rules

- Use one WebGL widget per dashboard state.
- Target roughly 100,000–300,000 rendered triangles for the initial model.
- Use compressed textures and mesh compression only after confirming loader support.
- Instantiate repeated markers instead of creating 60 unique meshes.
- Render labels in HTML or Babylon GUI; do not render every label continuously.
- Update materials only when severity changes, not on every sample.
- Visually scale shaft rotation rather than attempting literal 12,000-RPM rendering.
- Pause or reduce rendering when the widget is hidden.
- Host model and texture assets on a controlled HTTPS origin with correct CORS headers.

Credible animations

Animation	Data source	Recommendation
Inlet steam flow	FT_110A	Animated particles or UV motion when above a configured deadband
Leakage 1 flow	FT_162	Animated branch flow proportional to a clamped display scale
Leakage 2/3	No flow key shown	Directional symbolic animation only, or keep static
Turbine shaft	TURBINE_SPEED	Rotate at a perceptual scaled speed
Gearbox rotation	Gear ratio absent	Symbolic coupled motion; label as visualization
ESV/actuator	ACT_POS_FB	Animate valve/actuator opening percentage if geometry supports it
Demand versus feedback	HP_DEMAND, ACT_POS_FB	Two-position overlay and tracking-error chart
Temperature	TT/PYRO/RTD	Component outline or local heat halo, not fabricated CFD
Pressure	PT channels	Marker intensity or calibrated band color
Vibration	ZT/XT channels	Probe pulse/amplitude glyph; do not physically shake the entire model

Flow animation must stop or reverse only according to defined process logic. Negative calibrated pressure readings must not automatically reverse a flow path.

Advanced features

Feature	Within ThingsBoard	Recommendation
Realistic GLB model	Moderate	Build after SVG MVP
Transparent casing	Moderate	Use prepared transparent shell or material toggle
Cutaway	Moderate–difficult	Best authored as a separate model variant
Exploded view	Difficult	Animate predefined node transforms
Rotating shaft	Easy	Implement
Animated steam flow	Moderate	Implement for measured paths
Sensor markers	Easy	Essential
Floating labels	Easy–moderate	Essential; collision handling required
Dynamic component colors	Easy	Essential
Alarm pulses	Easy	Use sparingly
Vibration glyphs	Moderate	Implement without implying waveform
Temperature gradient	Moderate	Use local sensor-driven zones
Continuous CFD-like heatmap	Impractical without simulation	Avoid
Animated gearbox	Moderate	Symbolic unless ratio and geometry are known
Camera presets	Easy	Essential
Smooth camera transitions	Moderate	High visual payoff
Focus/isolate component	Moderate	Implement
Section clipping	Moderate–difficult	Advanced version
Synchronized chart selection	Moderate	Implement
Historical playback	Difficult	Add after timestamp correction
Fault localization	Moderate	Driven by alarm rules
Fault propagation	Difficult	Only with an explicit causal model
Predictive indicators	Difficult	Requires validated analytics, not decorative scores
True vibration orbit	Impractical with current CSV	Requires phase/synchronous waveform data

Visualization patterns to borrow

High-end industrial digital twins are most useful when 3D is treated as a navigation and spatial-context layer rather than as a replacement for trends and alarms. Reproduce these patterns:

- Contextual labels that appear only at relevant zoom levels.
- Camera presets for inlet, turbine, gearbox, dyno and leakage systems.
- Isolation mode that fades unrelated assemblies.
- Condition-based outlines instead of saturated whole-object coloring.

- A synchronized 2D/3D selection model.
- Component health based on the worst mapped sensor severity.
- Historical playback with a clear “replay” banner so historical state cannot be mistaken for live state.
- Side-by-side demand and feedback visualization.
- X/Y vibration pairing at each radial station.
- Data-quality and stale-data indication distinct from process alarms.
- Alarm-first navigation: clicking an alarm locates the sensor and opens its history.
- Cutaway or transparency as an intentional mode, not permanent visual clutter.

Avoid visually impressive but misleading features such as fictional blade-temperature fields, unmeasured steam velocity, fake shaft orbits and automatically inferred fault propagation.

Data architecture

Recommended ThingsBoard entity model:

- Asset: Steam Turbine Test Rig.
- Device: NI DAQ.
- Child assets: Inlet and Control, Turbine, Gearbox, Dyno, Leakage and Exhaust.
- Telemetry: normalized keys on the DAQ device.
- Shared attributes: units, engineering names, quality configuration, alarm limits and display precision.
- Widget metadata: sensor-to-component and 2D/3D position map.
- Alarms: generated from approved limits and validity logic.
- Relations: test rig → subsystem assets → DAQ device.

Normalize CSV headers during ingestion:

- PT_120 (kg/cm2) → PT_120
- PT_163 (kg/cm2) → PT_163
- XT_604 (mills) → XT_604
- XT_605 (mills) → XT_605
- XT_607 (mills) → XT_607
- PYRO_T (degC) → PYRO_T
- PYRO_GB (degC) → PYRO_GB
- ACT_POS_FB (%) → ACT_POS_FB

Retain the original header and source unit as metadata for traceability.

Delivery plan

MVP

Estimated effort: 3–5 developer-weeks, assuming the ThingsBoard instance and telemetry ingestion already work.

- Correct timestamp ingestion.
- Normalize telemetry keys.
- Produce an SVG version of both attached pages.
- Implement native/custom SVG interaction.
- Add markers and exact mappings for confirmed sensors.
- Add condition coloring and data-quality states.
- Create overview, process, turbovisory and sensor dashboard states.
- Add standard historical charts and alarm widgets.
- Keep unlocated sensors in a separate panel.

This version will already look substantially better than a gauge dashboard and remains technically defensible.

Advanced version

Estimated cumulative effort: 7–12 developer-weeks.

- Create or convert an optimized GLB model.
- Build the Babylon.js ThingsBoard widget.

- Add sensor markers, mesh labels, picking and camera presets.
- Animate measured flow paths, actuator position and shaft rotation.
- Synchronize 3D selection with native ThingsBoard charts.
- Add component isolation, transparency and condition halos.
- Add performance and actuator-tracking views.
- Harden loading, cleanup, resizing and fallback behavior.

Frontier version

Estimated effort: 3–6 months with 3D, ThingsBoard and rotating-equipment expertise.

- Accurate CAD-derived cutaway.
- Predefined exploded views.
- Historical state replay.
- Advanced clipping and section planes.
- Alarm-to-component fault localization.
- Operating-envelope visualization.
- Test-to-test comparison.
- Vibration spectrum and orbit modules after acquiring waveform, phase and tachometer data.
- Predictive-maintenance indicators backed by validated models.
- Optional external high-fidelity viewer synchronized with ThingsBoard through WebSocket/API integration.

Final build specification

The recommended production concept is:

- Babylon.js for the central 3D widget.
- GLB/gLTF for the equipment model.
- Native ThingsBoard SCADA SVG for the process-mimic and fallback view.
- FUXA patterns or components where they reduce SVG authoring effort.
- Native ThingsBoard time-series, alarm and action widgets for diagnostics.
- Dashboard states and entity aliases for drill-down.
- One metadata registry connecting telemetry keys, 2D coordinates, 3D positions, components, units and alarm limits.
- A strict engineering rule: no sensor is physically placed, derived KPI displayed, or effect animated unless supported by the PDF, CSV or subsequent instrumentation documentation.

The visually strongest first screen is therefore a neutral, realistic turbine train with contextual markers, a small number of permanent KPIs, animated measured flow, speed-driven shaft motion, alarm-based component outlines and one-click transitions into the exact process, vibration and sensor histories. The SVG view remains equally important: it provides the legible, deterministic operational representation that 3D alone cannot guarantee.

✱

1. <https://thingsboard.io/docs/pe/user-guide/advanced-guides-for-working-with-dashboard/2/>
2. 20260812_NL_DAQ_TEST_LOG_DigitalTwin.csv
3. UI_r6_DigitalTwin.pdf
4. <https://thingsboard.io/docs/user-guide/scada-symbol-dev/>
5. <https://thingsboard.io/docs/pe/reference/widgets/buttons/action-button/>
6. <https://thingsboard.io/docs/reference/widgets/cards/html-value-card/>
7. <https://thingsboard.io/docs/pe/user-guide/alarms/>
8. <https://thingsboard.io/docs/iot-hub/contribution-guides/widget/>
9. <https://thingsboard.io/docs/pe/user-guide/widgets/>
10. <https://thingsboard.io/docs/pe/user-guide/contribution/widgets-development/advanced/>
11. <https://thingsboard.io/docs/pe/user-guide/scada/>
12. <https://thingsboard.io/docs/pe/user-guide/csv-xls-data-export/>
13. <https://thingsboard.io/docs/pe/user-guide/dashboards/>
14. <https://thingsboard.io/docs/pe/reference/widgets/widget-library/>
15. <https://thingsboard.io/docs/releases/releases-table/v3-6-x/>
16. <https://thingsboard.io/docs/pe/reference/widgets/widget-api/>

17. <https://thingsboard.io/docs/reference/widgets/cards/html-card/>
18. <https://thingsboard.io/docs/pe/reference/widgets/video/configure/>
19. <https://thingsboard.io/docs/pe/user-guide/contribution/widgets-development/>
20. <https://thingsboard.io/docs/pe/user-guide/data-visualization/>
21. <https://thingsboard.io/docs/pe/reference/widgets/html-widgets/html-container/>
22. <https://thingsboard.io/docs/pe/user-guide/white-labeling-menu/>
23. <https://thingsboard.io/docs/pe/user-guide/contribution/widgets-development/static-widget/>
24. <https://thingsboard.io/docs/pe/reference/rest-api/>
25. <https://thingsboard.io/docs/pe/user-guide/reporting/dashboards/>
26. <https://thingsboard.io/docs/pe/user-guide/contribution/widgets-development/latest-values/>
27. <https://thingsboard.io/docs/trendz/embed-visuals/>
28. <https://thingsboard.io/docs/pe/user-guide/contribution/widgets-development/custom-subscription/>
29. <https://thingsboard.io/docs/pe/user-guide/contribution/widgets-development/time-series/>
30. <https://thingsboard.io/docs/pe/user-guide/advanced-guides-for-working-with-dashboard/1/>
31. <https://thingsboard.io/docs/pe/releases/releases-table/v3-2-x/>
32. <https://thingsboard.io/docs/pe/reference/widgets/scada/high-performance-fluid-system/hp-vertical-valve/>
33. <https://thingsboard.io/docs/pe/reference/widgets/maps/map/>
34. <https://thingsboard.io/docs/pe/reference/widgets/control-widgets/single-switch/>
35. <https://github.com/thingsboard/thingsboard/issues/5149>
36. <https://stackoverflow.com/questions/54651930/thingsboard-navigate-to-new-dashboard-state-widget-custom-action-javascript-code>
37. <http://www.ithingsboard.com/docs/pe/user-guide/scada/scada-symbols-dev-guide/>
38. <https://github.com/thingsboard/thingsboard/issues/10929>
39. <https://github.com/thingsboard/tbmq.io/blob/main/src/content/includes/docs/user-guide/scada-symbol-dev.mdx>
40. <https://github.com/thingsboard/tbmq.io/blob/main/src/content/docs/docs/paas/user-guide/scada-symbol-dev.mdx>
41. <https://hackmd.io/@Vian/Bkkz6TVK6>
42. <https://contextqmd.com/libraries/thingsboard/versions/4.3.1.1/pages/ui-ngx/src/app/modules/home/pages/scada-symbol/metadata-components/scada-symbol-behaviors.component>
43. <https://deepwiki.com/oliver225/thingsboard-ui-vue3/8.2-scada-symbols>
44. <https://thingsboard.io/docs/pe/user-guide/actions/>
45. <https://thingsboard.io/docs/pe/user-guide/scada-symbol-dev/>